FIT3003 Assignment 2

Data Warehouse Major Assignment

Student ID: 35947284

Student Name: Jiaxin Tian

C1

1. Project Overview and Data Warehouse Requirements

1.1 Project Background

MonCity operates a self-driving car system that generates operational data relating to booking, maintenance activities and accidents. The purpose of this project is to transform the existing operational data into a data warehouse that supports efficient management reporting and descriptive analysis.

Three major business processes are considered in the warehouse design: Booking, Maintenance and Accident. Attention is given to accident analysis at the individual self-driving car level because MonCity management is interested in identifying the number of accidents associated with specific cars.

1.2 Required Fact Measures

Business Processes	**Required Fact Measures**
Booking	Number of booking records
Maintenance	Number of maintenance records
Maintenance	Total maintenance cost
Accident	Number of accident records

1.3 Required Analytical Attributes

Area	Required Analysis
Booking	Month
Passenger	Faculty
Passenger	Age group
Car	Car body type/number of seats
Maintenance	Research centre
Maintenance	Maintenance type
Accident	Accident zone
Accident	Error code
Accident	Car damage severity
Accident	Registration number

C2(a)

2. Data Quality Analysis and Data Clean

2.1 Data Exploration Methodology

The source data in the MonCity database was examined to identify data quality issues before constructing the data warehouse. SQL queries were used to investigate missing values, invalid references, inconsistent values and duplicate records.

Issue	Table	Problem
1	PASSENGER	Invalid FACULTYID = Alienware
2	ACCIDENTINFO	Invalid ERRORCODE = ERROR010
3	ACCIDENTINFO	NULL ACCIDENTID
4	MAINTENANCE	MAINTENANCECOST = -200
5	BOOKING	Exact duplicate T1218

2.2 Issue 1 – Invalid Faculty reference

2.2.1 Issue Identification

A referential data quality problem was identified in the PASSENGER table. Passenger U163 has a FACULTYID value of Alienware. However, Alienware does not exist in the FACULTY table, whose valid identifiers are ART, BUS, ENG,

FIT, and SCI. Therefore, this record contains an invalid faculty reference.

2.2.2 Detection SQL

```
4  select p.*
5  from MONCITY.PASSENGER p
6  where p.FACULTYID not in (
7      select f.FACULTYID
8      from MONCITY.FACULTY f
9  );
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTORY TASK MONITOR

All rows fetched: 1 in 0.412 seconds

	PASSENGERID	PASSENGERNAME	PASSENGERROLE	PASSENGERGENDER	PASSENGERAGE	FACULTYID
1	U163	Anabia Mccabe	Staff	Male	21	Alienware

2.2.3 Support

```
469  select *
470    from moncity.faculty
471   order by facultyid;
472
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCR

All rows fetched: 5 in 0.227 seconds

	FACULTYID	FACULTYNAME	ZONE
1	ART	Art, Design and Architecture	ZoneD
2	BUS	Business and Economics	ZoneB
3	ENG	Engineering	ZoneC
4	FIT	Information Technology	ZoneA
5	SCI	Science	ZoneA

2.2.4 Cleaning Method and Justification

There is insufficient evidence in the source database to determine which valid faculty U163 actuality belongs to. Consequently, replacing Alienware with FIT, BUS, ENG, ART or SCI would require an unsupported assumption and could

introduce incorrect information into the warehouse.

Instead, the invalid faculty reference will be mapped to an Unknown/Unassigned faculty member during the ETL process. This approach preserves the passenger-related information while preventing the invalid source value from being treated as a valid faculty.

2.2.5 Cleaning Implementation

```
288  insert into FACULTYDIM (
289      FACULTYID,
290      FACULTYNAME,
291      ZONE
292  )
293  values (
294      'UNKNOWN',
295      'Unknown Faculty',
296      'Unknown'
297  );
```

```
295  select
296      p.passengerid,
297      p.facultyid as original_facultyid,
298      CASE
299          when f.facultyid is null then 'UNKNOWN'
300          ElSE p.facultyid
301      END as CLEANED_FACULTYID
302  from MonCity.PASSENGER p
303  LEFT JOIN MonCity.FACULTY f
304  on p.FACULTYID = f.FACULTYID
305  where p.passengerid = 'U163';
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT ···

All rows fetched: 1 in 0.214 seconds

	PASSENGERID	ORIGINAL_FACULTYID	CLEANED_FACULTYID
1	U163	Alienware	UNKNOWN

The transformation maps unmatched faculty references to the UNKNOWN member. For passenger U163, the original value Alienware is therefore transformed to UNKNOWN.

2.2.6 Validation

Validation was performed after implementing the cleaning rule. Alienware was not loaded as a valid member of FACULTYDIM, while the UNKNOWN member was successfully created for unresolved faculty references. An additional validation query confirmed that no unexpected invalid faculty identifiers remained in the dimension.

```
308   select *
309   from FACULTYDIM
310   where FACULTYID = 'Alienware';
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS QU

All rows fetched: 0 in 0.220 seconds

	FACULTYID	FACULTYNAME	ZONE

```
312   select *
313   from FACULTYDIM
314   where FACULTYID = 'UNKNOWN';
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUE

All rows fetched: 1 in 0.235 seconds

	FACULTYID	FACULTYNAME	ZONE
1	UNKNOWN	Unknown Faculty	Unknown

```
322   select count(*) as invalid_faculty_count
323     from facultydim d
324    where d.facultyid not in (
325       select facultyid
326         from moncity.faculty
327   )
328     and d.facultyid <> 'UNKNOWN';
329
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

All rows fetched: 1 in 0.220 seconds

	INVALID_FACULTY_COUNT
1	0

2.3 Issue 2 – Invalid Error reference

2.3.1 Issue Identification

A data quality issue was identified in the ACCIDENTINFO table. Accident A2000 contain an ERRORCODE value of Error010. However, no corresponding Error010 record exists in the ERROR table. The valid error codes contained in the source ERROR table are Error001 to Error005. Therefore, Error010 represents an invalid reference between ACCIDENTINFO and ERROR.

2.3.2 Detection SQL

```
138  select a.*
139  from MonCity.ACCIDENTINFO a
140  where a.ERRORCODE NOT IN
141  (
142      select e.ERRORCODE
143      from MonCity.ERROR e
144  );
145
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SQL HISTORY TASK MO

All rows fetched: 1 in 0.223 seconds

	ACCIDENTID	ACCIDENTZONE	CAR_DAMAGE_SEVERITY	ERRORCODE
1	A2000	ZoneB	No damage	Error010

2.3.3 Support

To verify the issue, the ERROR reference table was examined to determine the valid error codes.

```
478  select *
479    from moncity.error
480   order by errorcode;
481
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTO

All rows fetched: 5 in 0.286 seconds

	ERRORCODE	ERRORMESSAGE
1	Error001	Image recognition system failed
2	Error002	Low Battery
3	Error003	Signal loss: front sensor
4	Error004	Lidar systeem failed:Unable to locate
5	Error005	Unknow issue: please contact with maintenance team

2.3.4 Cleaning Method and Justification

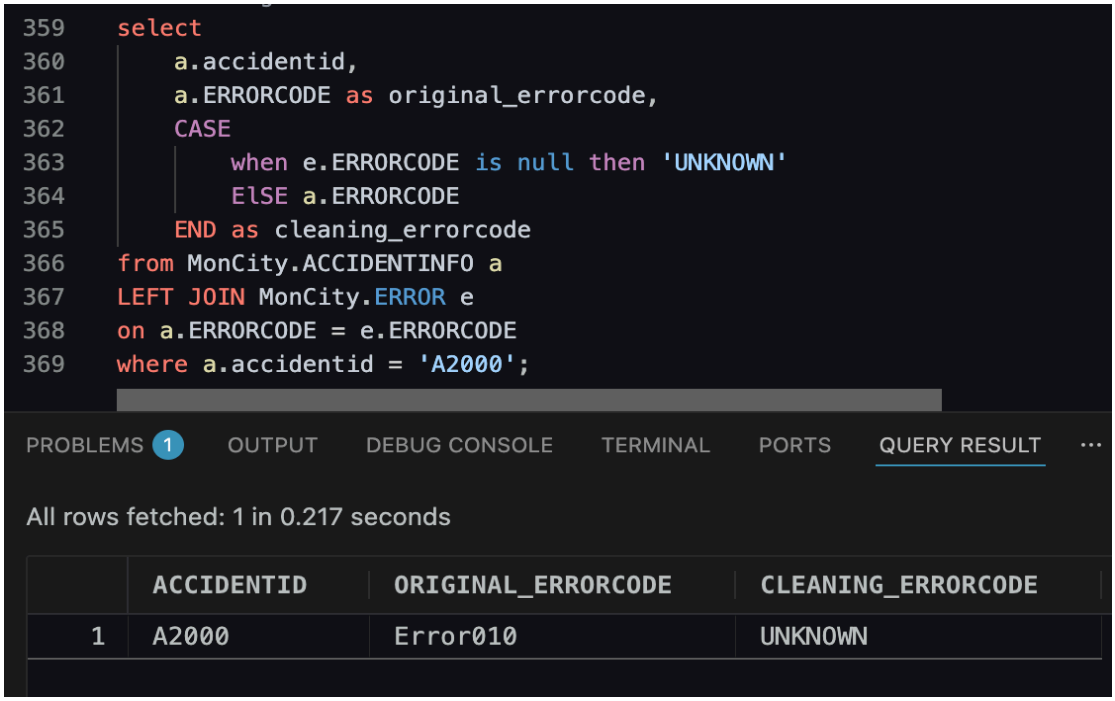
There is no reliable evidence in the source database indicating whatever Error010 should be Error001 to Error005.

Although the accident has a damage severity of No damage, damage severity does not establish the error that caused or was associated with the accident.

Therefore, the invalid error reference will be handled using an Unknown Error member during ETL.

2.3.5 Cleaning Implementation

```
347  insert into ERRORDIM (
348      ERRORCODE,
349      ERRORMESSAGE
350  )
351  values (
352      'UNKNOWN',
353      'Unknown Error'
354  );
355
```



The ETL transformation uses a LEFT JOIN between ACCIDENTINFO and ERROR. If no matching error record exists, the error reference is mapped to UNKNOWN. As a result, the invalid Error010 value associated with accident A2000 is transformed to UNKNOWN.

2.3.6 Validation

After implementing the cleaning rule, the UNKNOWN member was successfully created in ERRORDIM, while Error010 was not loaded as a valid error dimension member. A validation query was also used to check for any unexpected error codes in ERRORDIM. The result returned zero invalid error codes, confirming that all dimension members are either valid source error codes or the explicitly defined UNKNOWN

member.

```
379   select count(*) as invalid_error_count
380   from ERRORDIM d
381   where d.ERRORCODE not in (
382       select ERRORCODE
383       from MonCity.Error
384   )
385   and d.ERRORCODE <> 'UNKNOWN';
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

All rows fetched: 1 in 0.221 seconds

	INVALID_ERROR_COUNT
1	0

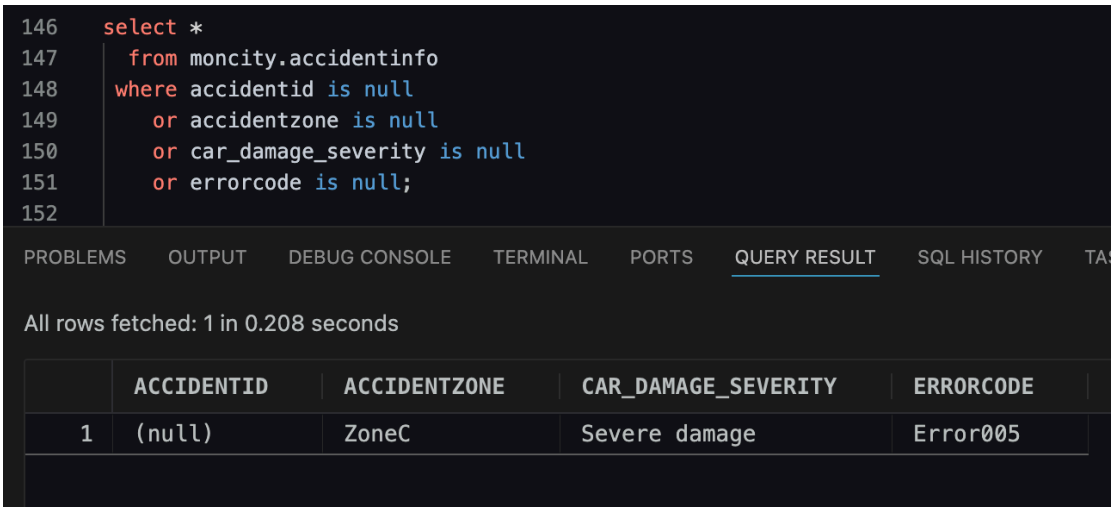
2.4 Issue 3 – Missing Accident Identifier

2.4.1 Issue Identification

A missing value data quality was identified in the ACCIDENTINFO table. One accident record contains a NULL value for ACCIDENTID.

An accident identifier is important for uniquely distinguishing accident events and become particularly relevant when accident information is associated with vehicles through CARACCIDENT.

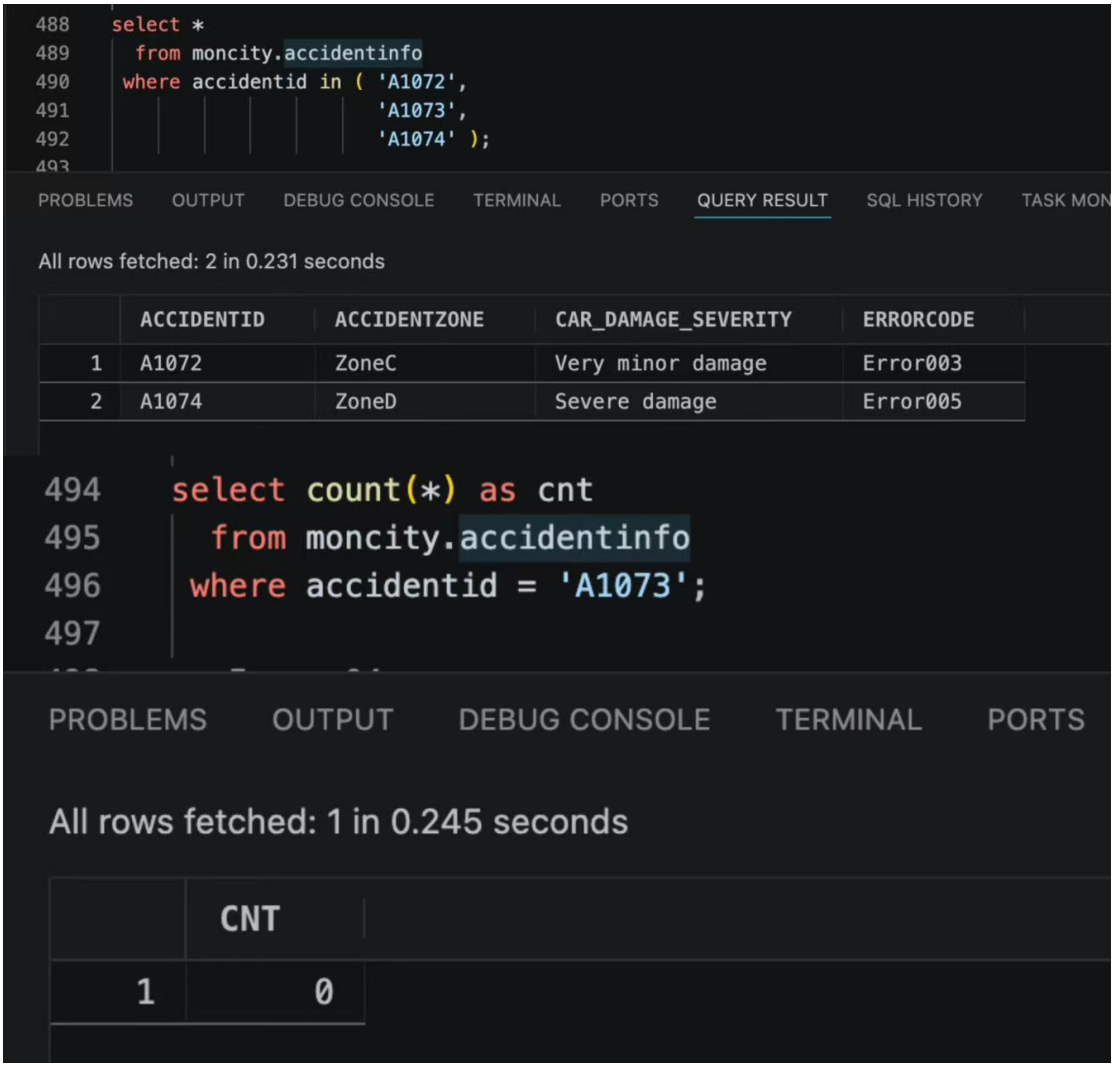
2.4.2 Detection SQL



2.4.3 Cleaning Method and Justification

Further investigation of the accident identifiers revealed a gap in the observed identifier sequence at A1073.

Therefore, although A1073 may appear to be a possible candidate based on the number sequence, there is insufficient source evidence to establish that the NULL identifier should be A1073.



```
488  select *
489    from moncity.accidentinfo
490   where accidentid in ( 'A1072',
491                         'A1073',
492                         'A1074' );
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SQL HISTORY TASK MON

All rows fetched: 2 in 0.231 seconds

	ACCIDENTID	ACCIDENTZONE	CAR_DAMAGE_SEVERITY	ERRORCODE
1	A1072	ZoneC	Very minor damage	Error003
2	A1074	ZoneD	Severe damage	Error005

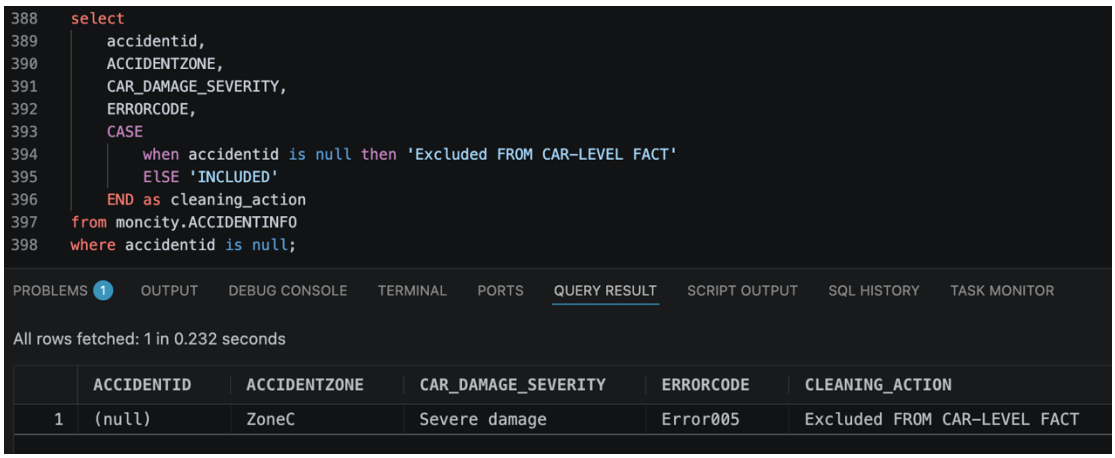
```
494  select count(*) as cnt
495    from moncity.accidentinfo
496   where accidentid = 'A1073';
497
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

All rows fetched: 1 in 0.245 seconds

	CNT
1	0

2.4.4 Cleaning Implementation



```
388  select
389      accidentid,
390      ACCIDENTZONE,
391      CAR_DAMAGE_SEVERITY,
392      ERRORCODE,
393      CASE
394          when accidentid is null then 'Excluded FROM CAR-LEVEL FACT'
395          ElSE 'INCLUDED'
396      END as cleaning_action
397  from moncity.ACCIDENTINFO
398  where accidentid is null;
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTORY TASK MONITOR

All rows fetched: 1 in 0.232 seconds

	ACCIDENTID	ACCIDENTZONE	CAR_DAMAGE_SEVERITY	ERRORCODE	CLEANING_ACTION
1	(null)	ZoneC	Severe damage	Error005	Excluded FROM CAR-LEVEL FACT

The cleaning rule identifies accident records with a missing ACCIDENTID and excludes them from the car-level fact

loading process. During the final ETL process, only accident records with a non-null identifier that can participate in the ACCIDENTINFO–CARACCIDENT relationship are used to construct the aggregated ACCIDENTFACT.

2.4.5 Validation

Validation confirmed that the accident record with the missing identifier cannot be matched to a car through CARACCIDENT. Therefore, excluding this record prevents an accident from being incorrectly attributed to a self-driving car and ensures that the car-level accident measures in the data warehouse are based only on identifiable source relationships.

```
401   select count(*) as loadable_null_accident_count
402   from MonCity.ACCIDENTINFO a
403   JOIN MonCity.caraccident ca
404   on a.accidentid = ca.accidentid
405   where a.accidentid is null;
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

All rows fetched: 1 in 0.230 seconds

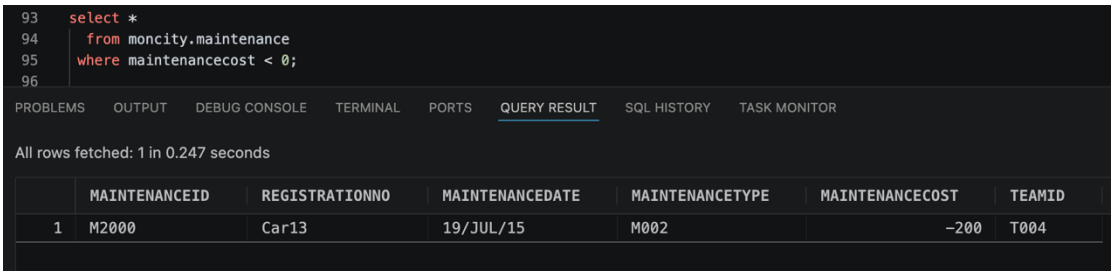
	LOADABLE_NULL_ACCIDENT_COUNT
1	0

2.5 Issue 4 – Invalid Negative Maintenance Cost

2.5.1 Issue Identification

An abnormal numerical value was identified in the MAINTENANCE table. Maintenance record M2000, with maintenance type M002, contains a MAINTENANCECOST value -200. A negative value is inconsistent with the maintenance cost values observed in the source data and was therefore investigated further.

2.5.2 Detection SQL



2.5.3 Support

```
460  select maintenancetype,
461         maintenancecost,
462         count(*) as cnt
463    from moncity.maintenance
464   group by maintenancetype,
465            maintenancecost
466   order by maintenancetype,
467            maintenancecost;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RE

All rows fetched: 6 in 0.545 seconds

	MAINTENANCETYPE	MAINTENANCECOST	CNT
1	M001	100	204
2	M002	-200	1
3	M002	200	197
4	M003	300	197
5	M004	400	172
6	M005	500	230

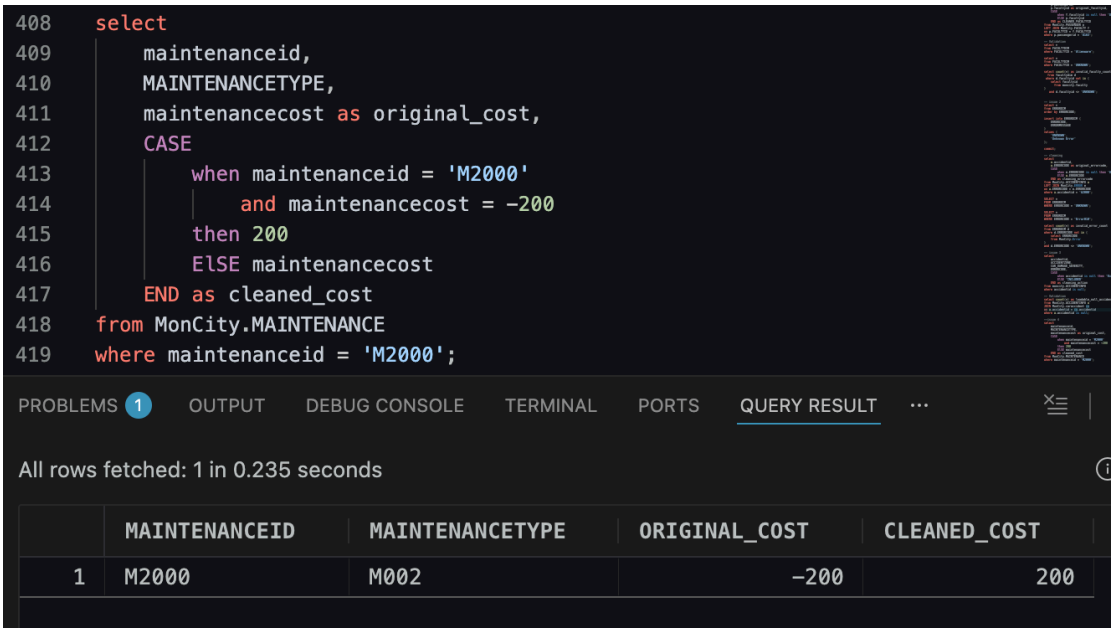
2.5.4 Cleaning Method and Justification

The value -200 is the only negative maintenance cost observed, more important, the remaining 197 records belonging to maintenance type M002 consistently have a cost of 200.

This provides strong evidence that the negative sign in maintenance record M2000 represents a data-entry error rather than a legitimate negative cost. The value will therefore be transformed from -200 to 200 during ETL.

2.5.5 Cleaning Implementation

A conditional ETL transformation was applied to M2000. The original cost of -200 is transformed to 200 before the maintenance costs are aggregated for the Version-1 fact table.



```
408  select
409      maintenanceid,
410      MAINTENANCETYPE,
411      maintenancecost as original_cost,
412      CASE
413          when maintenanceid = 'M2000'
414              and maintenancecost = -200
415          then 200
416          ElSE maintenancecost
417      END as cleaned_cost
418  from MonCity.MAINTENANCE
419  where maintenanceid = 'M2000';
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT ···

All rows fetched: 1 in 0.235 seconds

	MAINTENANCEID	MAINTENANCETYPE	ORIGINAL_COST	CLEANED_COST
1	M2000	M002	-200	200

2.5.6 Validation

Validation was performed on the transformed maintenance cost values. The query returned zero records with a negative cleaned maintenance cost, confirming that the anomalous negative value had been successfully corrected by the ETL transformation.

```
422  select count(*) as negative_cost_after_cleaning
423  from (
424      select
425          CASE
426              when maintenanceid = 'M2000'
427                  and maintenancecost = -200
428              then 200
429              ElSE maintenancecost
430          END as cleaned_cost
431      from MonCity.maintenance
432  )
433  where cleaned_cost < 0;
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

All rows fetched: 1 in 0.222 seconds

	NEGATIVE_COST_AFTER_CLEANING
1	0

2.6 Issue 5 – Duplicate Booking Record

2.6.1 Issue Identification

A duplicate transactional record was identified in BOOKING. Booking Id T1218 occurs twice with exactly the same attribute values.

2.6.2 Detection SQL

```sql
select bookingid,
       registrationno,
       bookingdate,
       departurezone,
       destinationzone,
       passengerid,
       count(*) as duplicate_count
  from moncity.booking
 group by bookingid,
          registrationno,
          bookingdate,
          departurezone,
          destinationzone,
          passengerid
having count(*) > 1;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SQL HISTORY TASK MONITOR

All rows fetched: 1 in 0.531 seconds

	BOOKINGID	REGISTRATIONNO	BOOKINGDATE	DEPARTUREZONE	DESTINATIONZONE	PASSENGERID	DUPLICATE_COUNT
1	T1218	Car14	29/JUL/18	ZoneD	ZoneB	U059	2

2.6.3 Support

```sql
SELECT *
FROM MonCity.BOOKING
WHERE BOOKINGID = 'T1218';
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SQL HISTORY TASK MONITOR

All rows fetched: 2 in 0.282 seconds

	BOOKINGID	REGISTRATIONNO	BOOKINGDATE	DEPARTUREZONE	DESTINATIONZONE	PASSENGERID
1	T1218	Car14	29/JUL/18	ZoneD	ZoneB	U059
2	T1218	Car14	29/JUL/18	ZoneD	ZoneB	U059

2.6.4 Cleaning Method and Justification

Because the two records are identical across the booking attributes, one occurrence is redundant. The duplicate will therefore be removed logically during ETL.

2.6.5 Cleaning Implementation

```sql
select distinct
    BOOKINGID,
    REGISTRATIONNO,
    BOOKINGDATE,
    DEPARTUREZONE,
    DESTINATIONZONE,
    passengerid
from MonCity.BOOKING;
```

The cleaning transformation applies DISTINCT to the complete booking record before the data is joined with the dimensions and aggregated into BOOKINGFACT. For booking T1218, the two identical source rows are reduced to one unique booking record.

```sql
select *
from (
    select distinct
        BOOKINGID,
        REGISTRATIONNO,
        BOOKINGDATE,
        DEPARTUREZONE,
        DESTINATIONZONE,
        passengerid
    from MonCity.BOOKING
)
where BOOKINGID = 'T1218';
```

PROBLEMS 1 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | QUERY RESULT | SCRIPT OUTPUT | SQL HISTORY | TASK MONITOR

All rows fetched: 1 in 0.232 seconds

	BOOKINGID	REGISTRATIONNO	BOOKINGDATE	DEPARTUREZONE	DESTINATIONZONE	PASSENGERID
1	T1218	Car14	29/JUL/18	ZoneD	ZoneB	U059

2.6.6 Validation

Validation was performed by comparing the number of occurrences of T1218 before and after the deduplication transformation. The source table contained two identical occurrences, whereas the transformed dataset contained only

one.

```
459  select (
460      select count(*)
461      from MonCity.BOOKING
462      where BOOKINGID = 'T1218'
463  ) as before_cleaning,
464
465      (
466          select count(*)
467          from (
468              select distinct
469                  BOOKINGID,
470                  REGISTRATIONNO,
471                  BOOKINGDATE,
472                  DEPARTUREZONE,
473                  DESTINATIONZONE,
474                  passengerid
475              from MonCity.BOOKING
476          )
477          where BOOKINGID = 'T1218'
478      ) as after_cleaning
479  from DUAL;
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

All rows fetched: 1 in 0.230 seconds

	BEFORE_CLEANING	AFTER_CLEANING
1	2	1

C3(b)

3. Version 1 star/snowflake schema

3.1 Requirements-To-Schema Analysis

Required Query	**Dimension Attributes Needed**	**Measure**
IT Faculty + July + Bus	Faculty, Month, Car body type	Booking count
Each Passenger Age Group	Age group	Booking count
Maintenance Involving CE04	Research center	Maintenance count
Each Maintenance Type + Mini bus	Maintenance type, Car body type	Maintenance cost
ZoneA + Car01	Accident zone, Registration no.	Accident count
Each Error Code + ZoneB + Car06	Error code, Accident zone, Registration No.	Accident count
Severity Damage + Car05	Car damage severity, Registration No.	Accident count

3.2 Fact Measure

BOOKINGFACT: *NUMBER_OF_BOOKINGS*

MAINTENANCEFACT: *NUMBER_OF_MAINTENANCES, TOTAL_MAINTENANCE_COST*

ACCIDENTFACT: *NUMBER_OF_ACCIDENTS*

3.3 Version 1 Fact Granularity

3.3.1 BOOKINGFACT

The version 1 BOOKINGFACT is aggregated by:

Booking Month × Faculty × Passenger Age Group × Car Body Type

This is the highest useful aggregation because the required booking queries do not need individual booking IDs, individual passengers, or individual cars.

3.3.2 MAINTENANCEFACT

The MAINTENANCEFACT must support analysis by:

Car Body Type × Maintenance Type × Maintenance Team

(A complication exists because maintenance records identify a

maintenance team, while the BELONG table indicates that a maintenance team may belong to many research centers.)

3.3.3 ACCIDENTFACT

The required accident queries explicitly reference individual cars such as Car01, Car02, Car03 etc. Consequently, unlike the booking fact, the accident fact cannot be aggregated only by car body type.

The Version 1 accident grain is:

Registration No. × Accident Zone × Error Code × Car Damage Severity

3.4 Dimension Design

3.4.1 Time Dimension

The Time dimension supports the required booking analysis by month. Because Version 1 booking analysis only requires monthly granularity, unnecessary day-level detail is not required for the aggregated booking fact.

Relevant attributes include:

MONTH, YEAR

The inclusion of Year prevents booking from the same month

name across different years from being incorrect combined.

3.4.2 Faculty Dimension

The Faculty dimension represents the passenger's faculty and supports queries such as booking made by passengers from the IT faculty.

Relevant attributes include:

FACULTYID, FACULTYNAME, ZONE

An Unknown member will also support the cleaning of invalid faculty references identified in Section 2.

3.4.3 Age Group Dimension

The Age Group dimension represents the categories explicitly specified by the assignment:

Young adults: 18-35

Middle-aged adults: 36-59

Old-aged adults: 60 and older

The warehouse stores the category rather than requiring the analytical query to repeatedly calculate age ranges from individual passenger ages.

3.4.4 Car Dimension

The required analysis uses two different levels of car information.

Booking and maintenance questions require car body type and number of seats, whereas accident questions require identification of individual cars (Registration No.).

3.4.5 Research Center Dimension

The Research Center dimension supports the required analysis of maintenance records associated with research center CE04.

Relevant attributes include:

CENTERID, CENTERNAME, PHONE, OPENINGHOUR

Because a maintenance team may belong to multiple research centers, this dimension participates in the maintenance analysis through the final bridge-table design.

3.4.6 Maintenance Type Dimension

The Maintenance Type dimension supports maintenance cost analysis by maintenance type and is derived from MAINTENANCETYPE.

Relevant attributes include:

MAINTENANCETYPE, MAINTENANCEDESCRIPTION

3.4.7 Maintenance Team Dimension

The MAINTENANCETEAMDIM represents the maintenance teams responsible for maintenance activities. It is derived from the MAINTENANCETEAM source table and contains:

TEAMID, TEAMLEADER, PHONE, ALLOCATION_WEIGHT, CENTERGROUPLIST

3.4.8 Error Dimension

The Error dimension supports grouping accident records according to error code.

Relevant attributes include:

ERRORCODE, ERRORMESSAGE

An Unknown Error member will also allow the invalid Error010 source value to be handled.

3.5 Bridge Table, Weight Fact, and List Aggregate

A bridge table is used to represent the many-to-many relationship between maintenance teams and research centers. In the operational database, a maintenance team can belong to multiple research centers, while a research center can also contain multiple maintenance teams. Therefore, RESEARCHCENTERDIM should not be connected directly to MAINTENANCEFACT.

Instead, MAINTENANCETEAMDIM is connected to MAINTENANCEFACT, and MAINTENANCE_TEAM_BRIDGE links MAINTENANCETEAMDIM with RESEARCHCENTERDIM.

The bridge table contains TeamID and CenterID, which identify the associations between maintenance teams and research centers.

A weight factor, Allocation_Weight, is included in MAINTENANCETEAMDIM. For a maintenance team associated with n research centers, the weight factor is calculated as:

Allocated_Weight = 1/n

For example, T001 belongs to four research centers (CE01, CE02, CE03, and CE04), so its allocation weight is 1/4 = 0.25. The weight factor can be used when estimating the proportional contribution of each research center to maintenance measures.

A list aggregate attribute, CenterGroupList, is also included in MAINTENANCETEAMDIM. It stores the complete group of research centers associated with each maintenance team as a concatenated list. For example, the CenterGroupList for T001 is CE01_CE02_CE03_CE04. This provides a direct representation of the complete research-center group associated with each

maintenance team.

3.6 Version 1 Schema Design

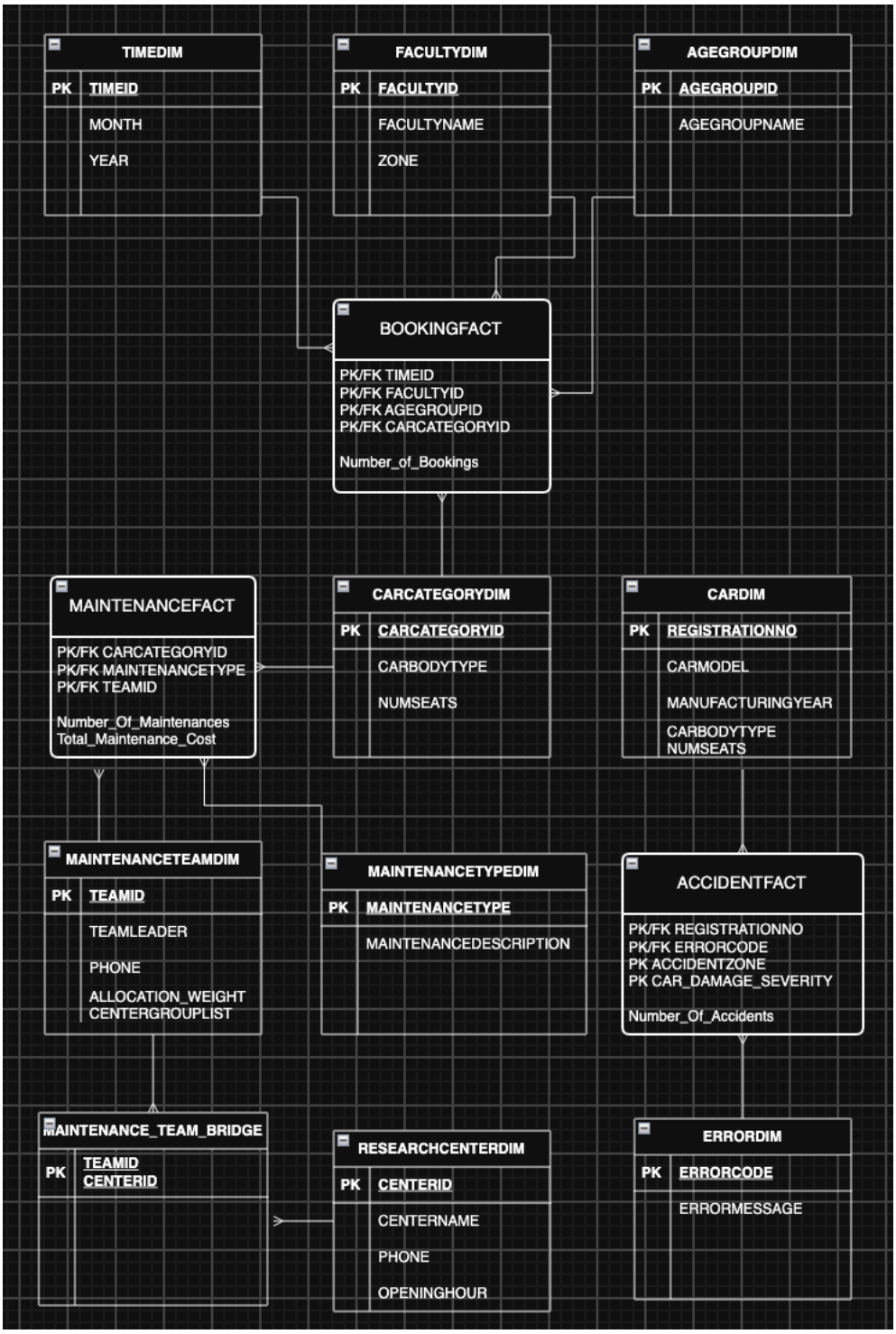
Design Version 1 uses a multi-fact dimensional model containing Booking, Maintenance and Accident fact tables. The grain of each fact table was selected at the highest level of aggregation that still supports the seven analytical queries specified in the assignment.

BOOKINGFACT is aggregated by booking month, faculty, passenger age group and car category because the required booking queries do not require individual booking, passenger or vehicle-level detail. MAINTENANCEFACT is aggregated by car category, maintenance type and maintenance team. Maintenance team is retained because research-center membership is determined through the relationship between maintenance teams and research centers. A bridge table is used to represent the many-to-many relationship between maintenance teams and research centers.

ACCIDENTFACT retains individual vehicle registration number as part of its grain because the required accident queries explicitly refer to individual vehicles including Car01, Car05 and Car06. The remaining accident analysis is performed by error code,

accident zone and car damage severity.

Two levels of car information are therefore represented in the Version-1 design. CARCATEGORYDIM supports the highly aggregated Booking and Maintenance facts using car body type and number of seats, whereas CARDIM retains individual vehicle information required by the Accident fact. This avoids unnecessarily reducing the aggregation level of Booking and Maintenance while preserving the individual-car detail explicitly required for accident analysis.


TIMEDIM
PK TIMEID
MONTH
YEAR
FACULTYDIM
PK FACULTYID
FACULTYNAME
ZONE
AGEGROUPDIM
PK AGEGROUPID
AGEGROUPNAME
BOOKINGFACT
PK/FK TIMEID
PK/FK FACULTYID
PK/FK AGEGROUPID
PK/FK CARCATEGORYID
Number_of_Bookings
MAINTENANCEFACT
PK/FK CARCATEGORYID
PK/FK MAINTENANCETYPE
PK/FK TEAMID
Number_Of_Maintenances
Total_Maintenance_Cost
CARCATEGORYDIM
PK CARCATEGORYID
CARBODYTYPE
NUMSEATS
CARDIM
PK REGISTRATIONNO
CARMODEL
MANUFACTURINGYEAR
CARBODYTYPE
NUMSEATS
MAINTENANCETEAMDIM
PK TEAMID
TEAMLEADER
PHONE
ALLOCATION_WEIGHT
CENTERGROUPLIST
MAINTENANCETYPEDIM
PK MAINTENANCETYPE
MAINTENANCEDESCRIPTION
ACCIDENTFACT
PK/FK REGISTRATIONNO
PK/FK ERRORCODE
PK ACCIDENTZONE
PK CAR_DAMAGE_SEVERITY
Number_Of_Accidents
MAINTENANCE_TEAM_BRIDGE
PK TEAMID
CENTERID
RESEARCHCENTERDIM
PK CENTERID
CENTERNAME
PHONE
OPENINGHOUR
ERRORDIM
PK ERRORCODE
ERRORMESSAGE

C3(c)

3.7 Design Version 2 – Level 0 Granularity

Design Version 1 uses highly aggregated fact tables to support the analytical queries specified in the assignment. For Design Version 2, the granularity of the fact tables can be increased to Level 0 (no aggregation) by retaining the identifiers and attributes of individual operational events. At level 0, each fact row represents the lowest level business event rather than an aggregated group of records.

BOOKINGFACT

In Version 1, BOOKINGFACT is aggregated by booking month, faculty, passenger age group and car category. To achieve Level 0 in Version 2, each row should represent one individual booking transaction. BookingID should therefore be added to identify each booking record. Transaction level attributes including PassengerID, RegistrationNo, BookingDate, DepartureZone, and DestinationZone should also be retained. Number_Of_Bookings would no longer need to be stored as a pre-aggregated measure because the number of bookings can be calculated by counting individual booking records.

MAINTENANCEFACT

In Version1, MAINTENANCEFACT is aggregated by car category, maintenance type and maintenance team, with Number_Of_Maintenance and Total_Maintenance_Cost stored as aggregated measures. To achieve Level 0, each fact row should represent one individual maintenance record. MaintenanceID, RegistrationNo, and MaintenanceDate should therefore be added. MaintenanceCost should be retained at the individual maintenance record level instead of storing only Total_Maintenance_Cost. The number of maintenance records and total maintenance cost can subsequently be calculated using COUNT and SUM.

ACCIDENTFACT

In Version 1, ACCIDENTFACT is aggregated by vehicle, error codes, accident zone and car damage severity. To achieve Level0, AccidentID should be added. Since the operational database specifies that one accident can involve more than one car, the lowest useful grain is one row per accident-car combination rather than simply one row per accident. Therefore, AccidentID and RegistrationNo should be retained together with ErrorCode, AccidentZone, and Car_Damage_severity.
Number_Of_Accidents should no longer need to be stored as a pre-aggregated measure, because accident counts can be

calculated from the Level 0 records.

Fact Table	Version-1 Grain	Version-2 Level 0 Grain	Main Attributes Changed / Added
`BOOKINGFACT`	Month × Faculty × Age Group × Car Category	One row per booking transaction	Add `BookingID`, `PassengerID`, `RegistrationNo`, `BookingDate`, `DepartureZone`, `DestinationZone`; remove pre-aggregated `Number_Of_Bookings`
`MAINTENANCEFACT`	Car Category × Maintenance Type × Maintenance Team	One row per maintenance record	Add `MaintenanceID`, `RegistrationNo`, `MaintenanceDate`; replace `Total_Maintenance_Cost` with transaction-level `MaintenanceCost`; `Number_Of_Maintenance` no longer needs to be pre-aggregated
`ACCIDENTFACT`	Car × Error Code × Accident Zone × Damage Severity	One row per accident–car combination	Add `AccidentID`; retain `RegistrationNo`, `ErrorCode`, `AccidentZone`, `Car_Damage_Severity`; `Number_Of_Accidents` is no longer pre-aggregated

C3(d)

3.8 Slowly Changing Dimensions and Temporal Dimension

3.8.1 Slowly Changing Dimension

SCD are used to manage dimension records whose attributes values change slowly over time. Different SCD types provide different approaches to preserving original, current, and historical dimension values.

SCD Type 0 – Original Value

Type 0 preserves the original or initial value of a dimension record. Subsequent changes to the attribute are not reflected in the stored dimension value. Therefore, historical changes are not recorded.

SCD Type 1 - Latest Value

Type 1 overwrites the existing dimension attributes with its latest value. The previous value is lost and the history of changes cannot be reconstructed.

SCD Type 2 – Complete History

Type 2 preserves the history of dimension changes by adding a new dimension record. Different versions of the same business entity can therefore represent different periods.

SCD Type 3 – Limited History

Type 3 stores a limited history value using Additional attributes, such as the current and previous values, within the dimension.

SCD Type 4 – Separate Historical Dimension

Type 4 separates changing historical attributes from the main dimension. A separate dimension is created to maintain the history of the changing attributes, together with temporal information.

SCD Type 6 – Combination of Type 2 and Type 3

Type 6 preserves the complete history while also supporting attributes representing current and historical values.

SCD Type	How a change is handled	History retained	Main idea
0	Keep original value	No	Original value
1	Overwrite with latest value	No	Latest value
2	Add new	Complete	Multiple

	dimension record		historical records
3	Store limited old	Limited	Previous/current attributes
4	Store history in separate dimension	Yes	Separate historical dimension
6	Combination of Type 2 and Type 3	Complete	Historical records + current/historical attributes

3.8.2 Proposed Temporal Dimension for MonCity

A temporal dimension is a dimension in which records have a lifetime or period of validity. For MonCity, FACULTYDIM can be treated as a temporal dimension in Design Version 2 if the zone associated with a faculty change over time.

For example, assume that the Faculty of Information Technology (FIT) is associated with ZoneA during one period and subsequently changes to ZoneB. Simply overwriting ZoneA with ZoneB would cause the previous faculty-zone

relationship to be lost and could affect historical analysis of booking records.

Therefore, SCD Type 2 is proposed for FACULTYDIM in Design Version 2. When the zone of a faculty changes, a new version of the faculty record would be created instead of overwriting the previous record. Each version can be associated with its applicable period, allowing historical booking records to be analysed using the faculty information that was valid at the relevant time.

FacultyVersion	FacultyID	FacultyName	Zone	StartDate	EndDate
1	FIT	Information Technology	ZoneA	01/01/2015	31/12/2018
2	FIT	Information Technology	ZoneB	01/01/2019	Current

The values and dates above are illustrative examples used to demonstrate the temporal design and do not represent observed changes in the MonCity source data.

3.8.3 Justification of SCD

SCD Type 2 is preferred because the complete history of faculty-zone changes can be retained. Type 0 retain only the

original zone, Type 1 retain only the latest zone and lost historical information. Type 3 preserve only limited history and may become insufficient if the faculty changes zone multiple time. Type 4 also preserve the history using a separate historical dimension, but this would introduce an additional structure. Type 6 provides complete historical capability but is more complex than required for this example. Type 2 provides an appropriate balance between complete historical preservation and design simplicity for the FACULTYDIM.

C4

Dimension/Bridge Structures

```
SELECT
    TABLE_NAME,
    COLUMN_NAME,
    DATA_TYPE,
    DATA_LENGTH,
    NULLABLE
FROM USER_TAB_COLUMNS
WHERE TABLE_NAME IN (
    'TIMEDIM',
    'FACULTYDIM',
    'AGEGROUPDIM',
    'CARCATEGORYDIM',
    'CARDIM',
    'MAINTENANCETEAMDIM',
    'MAINTENANCETYPEDIM',
    'RESEARCHCENTERDIM',
    'ERRORDIM',
    'MAINTENANCE_TEAM_BRIDGE'
)
ORDER BY TABLE_NAME, COLUMN_ID;
```

All rows fetched: 31 in 0.405 seconds

	TABLE_NAME	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	NULLABLE
1	AGEGROUPDIM	AGEGROUPID	VARCHAR2	10	N
2	AGEGROUPDIM	AGEGROUPNAME	VARCHAR2	50	N
3	CARCATEGORYDIM	CARCATEGORYID	VARCHAR2	10	N
4	CARCATEGORYDIM	CARBODYTYPE	VARCHAR2	50	N
5	CARCATEGORYDIM	NUMSEATS	NUMBER	22	N
6	CARDIM	REGISTRATIONNO	VARCHAR2	20	N
7	CARDIM	CARMODEL	VARCHAR2	100	Y
8	CARDIM	MANUFACTURINGYEAR	NUMBER	22	Y
9	CARDIM	CARBODYTYPE	VARCHAR2	50	N
10	CARDIM	NUMSEATS	NUMBER	22	N
11	ERRORDIM	ERRORCODE	VARCHAR2	20	N
12	ERRORDIM	ERRORMESSAGE	VARCHAR2	200	Y
13	FACULTYDIM	FACULTYID	VARCHAR2	20	N
14	FACULTYDIM	FACULTYNAME	VARCHAR2	100	N
15	FACULTYDIM	ZONE	VARCHAR2	20	Y
16	MAINTENANCETEAMDIM	TEAMID	VARCHAR2	20	N
17	MAINTENANCETEAMDIM	TEAMLEADER	VARCHAR2	100	Y
18	MAINTENANCETEAMDIM	PHONE	NUMBER	22	Y
19	MAINTENANCETEAMDIM	ALLOCATION_WEIGHT	NUMBER	22	Y
20	MAINTENANCETEAMDIM	CENTERGROUPLIST	VARCHAR2	200	Y
21	MAINTENANCETYPEDIM	MAINTENANCETYPE	VARCHAR2	20	N
22	MAINTENANCETYPEDIM	MAINTENANCEDESCRIPTION	VARCHAR2	200	Y
23	MAINTENANCE_TEAM_BRIDGE	TEAMID	VARCHAR2	20	N
24	MAINTENANCE_TEAM_BRIDGE	CENTERID	VARCHAR2	20	N
25	RESEARCHCENTERDIM	CENTERID	VARCHAR2	20	N
26	RESEARCHCENTERDIM	CENTERNAME	VARCHAR2	150	N
27	RESEARCHCENTERDIM	PHONE	NUMBER	22	Y
28	RESEARCHCENTERDIM	OPENINGHOUR	VARCHAR2	50	Y
29	TIMEDIM	TIMEID	VARCHAR2	6	N
30	TIMEDIM	MONTH	NUMBER	22	N
31	TIMEDIM	YEAR	NUMBER	22	N

Fact Structures

```
536  SELECT
537      TABLE_NAME,
538      COLUMN_NAME,
539      DATA_TYPE,
540      DATA_LENGTH,
541      NULLABLE
542  FROM USER_TAB_COLUMNS
543  WHERE TABLE_NAME IN (
544      'BOOKINGFACT',
545      'MAINTENANCEFACT',
546      'ACCIDENTFACT'
547  )
548  ORDER BY TABLE_NAME, COLUMN_ID;
549
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTORY TASK MONITOR

All rows fetched: 15 in 0.423 seconds

	TABLE_NAME	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	NULLABLE
1	ACCIDENTFACT	REGISTRATIONNO	VARCHAR2	20	N
2	ACCIDENTFACT	ERRORCODE	VARCHAR2	20	N
3	ACCIDENTFACT	ACCIDENTZONE	VARCHAR2	20	N
4	ACCIDENTFACT	CAR_DAMAGE_SEVERITY	VARCHAR2	50	N
5	ACCIDENTFACT	NUMBER_OF_ACCIDENTS	NUMBER	22	N
6	BOOKINGFACT	TIMEID	VARCHAR2	6	N
7	BOOKINGFACT	FACULTYID	VARCHAR2	20	N
8	BOOKINGFACT	AGEGROUPID	VARCHAR2	10	N
9	BOOKINGFACT	CARCATEGORYID	VARCHAR2	10	N
10	BOOKINGFACT	NUMBER_OF_BOOKINGS	NUMBER	22	N
11	MAINTENANCEFACT	CARCATEGORYID	VARCHAR2	10	N
12	MAINTENANCEFACT	MAINTENANCETYPE	VARCHAR2	20	N
13	MAINTENANCEFACT	TEAMID	VARCHAR2	20	N
14	MAINTENANCEFACT	NUMBER_OF_MAINTENANCE	NUMBER	22	N
15	MAINTENANCEFACT	TOTAL_MAINTENANCE_COST	NUMBER	22	N

PK/FK Constraints

```sql
SELECT
    TABLE_NAME,
    CONSTRAINT_NAME,
    CONSTRAINT_TYPE,
    STATUS
FROM USER_CONSTRAINTS
WHERE TABLE_NAME IN (
    'TIMEDIM',
    'FACULTYDIM',
    'AGEGROUPDIM',
    'CARCATEGORYDIM',
    'CARDIM',
    'MAINTENANCETEAMDIM',
    'MAINTENANCETYPEDIM',
    'RESEARCHCENTERDIM',
    'ERRORDIM',
    'MAINTENANCE_TEAM_BRIDGE',
    'BOOKINGFACT',
    'MAINTENANCEFACT',
    'ACCIDENTFACT'
)
AND CONSTRAINT_TYPE IN ('P', 'R')
ORDER BY TABLE_NAME, CONSTRAINT_TYPE;
```

All rows fetched: 22 in 2.566 seconds

	TABLE_NAME	CONSTRAINT_NAME	CONSTRAINT_TYPE	STATUS
1	ACCIDENTFACT	PK_ACCIDENTFACT	P	ENABLED
2	ACCIDENTFACT	FK_ACCIDENT_CAR	R	ENABLED
3	ACCIDENTFACT	FK_ACCIDENT_ERROR	R	ENABLED
4	AGEGROUPDIM	SYS_C00121270	P	ENABLED
5	BOOKINGFACT	PK_BOOKINGFACT	P	ENABLED
6	BOOKINGFACT	FK_BOOKING_AGEGROUP	R	ENABLED
7	BOOKINGFACT	FK_BOOKING_TIME	R	ENABLED
8	BOOKINGFACT	FK_BOOKING_CARCATEGORY	R	ENABLED
9	CARCATEGORYDIM	SYS_C00121273	P	ENABLED
10	CARDIM	SYS_C00121276	P	ENABLED
11	ERRORDIM	SYS_C00121281	P	ENABLED
12	FACULTYDIM	SYS_C00122871	P	ENABLED
13	MAINTENANCEFACT	PK_MAINTENANCEFACT	P	ENABLED
14	MAINTENANCEFACT	FK_MAINTENANCE_CARCATEGORY	R	ENABLED
15	MAINTENANCEFACT	FK_MAINTENANCE_TEAM	R	ENABLED
16	MAINTENANCETEAMDIM	SYS_C00121278	P	ENABLED
17	MAINTENANCETYPEDIM	SYS_C00121277	P	ENABLED
18	MAINTENANCE_TEAM_BRIDGE	PK_MAINTENANCE_TEAM_BRIDGE	P	ENABLED
19	MAINTENANCE_TEAM_BRIDGE	FK_BRIDGE_CENTER	R	ENABLED
20	MAINTENANCE_TEAM_BRIDGE	FK_BRIDGE_TEAM	R	ENABLED
21	RESEARCHCENTERDIM	SYS_C00121280	P	ENABLED
22	TIMEDIM	SYS_C00121220	P	ENABLED

Row-count

```sql
SELECT 'TIMEDIM' AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM TIMEDIM
UNION ALL
SELECT 'FACULTYDIM', COUNT(*) FROM FACULTYDIM
UNION ALL
SELECT 'AGEGROUPDIM', COUNT(*) FROM AGEGROUPDIM
UNION ALL
SELECT 'CARCATEGORYDIM', COUNT(*) FROM CARCATEGORYDIM
UNION ALL
SELECT 'CARDIM', COUNT(*) FROM CARDIM
UNION ALL
SELECT 'MAINTENANCETEAMDIM', COUNT(*) FROM MAINTENANCETEAMDIM
UNION ALL
SELECT 'MAINTENANCETYPEDIM', COUNT(*) FROM MAINTENANCETYPEDIM
UNION ALL
SELECT 'RESEARCHCENTERDIM', COUNT(*) FROM RESEARCHCENTERDIM
UNION ALL
SELECT 'ERRORDIM', COUNT(*) FROM ERRORDIM
UNION ALL
SELECT 'MAINTENANCE_TEAM_BRIDGE', COUNT(*) FROM MAINTENANCE_TEAM_BRIDGE;
```

All rows fetched: 10 in 0.223 seconds

	TABLE_NAME	ROW_COUNT
1	TIMEDIM	84
2	FACULTYDIM	6
3	AGEGROUPDIM	3
4	CARCATEGORYDIM	3
5	CARDIM	30
6	MAINTENANCETEAMDIM	5
7	MAINTENANCETYPEDIM	5
8	RESEARCHCENTERDIM	4
9	ERRORDIM	6
10	MAINTENANCE_TEAM_BRIDGE	12

```
513    SELECT 'BOOKINGFACT' AS TABLE_NAME,
514           COUNT(*) AS ROW_COUNT
515    FROM BOOKINGFACT
516    UNION ALL
517    SELECT 'MAINTENANCEFACT',
518           COUNT(*)
519    FROM MAINTENANCEFACT
520    UNION ALL
521    SELECT 'ACCIDENTFACT',
522           COUNT(*)
523    FROM ACCIDENTFACT;
524
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL P

All rows fetched: 3 in 0.230 seconds

	TABLE_NAME	ROW_COUNT
1	BOOKINGFACT	2853
2	MAINTENANCEFACT	75
3	ACCIDENTFACT	550

C5 Data Analysis – 484 words

Overview

The implemented Version-1 data warehouse was used to analyse MonCity's booking demand, vehicle maintenance expenditure and accident records. The analysis covers 10,000 booking records, 1,001 maintenance records and 1,000 car-accident records.

Finding 1: Booking Demand by Car Body Type

Booking demand is relatively evenly distributed across the three vehicle categories. People Movers recorded the highest number of bookings at 3,396 (33.96%), followed by Mini Buses with 3,313 (33.13%) and Buses with 3,291 (32.91%).

The difference between the highest and lowest categories is only 105 bookings, suggesting that all three vehicle types contribute substantially to MonCity's operations. This supports maintaining a balanced fleet rather than prioritising one category solely based on booking frequency.

Finding 2: Maintenance Expenditure by Vehicle Category

Total maintenance expenditure across 1,001 maintenance records

was 302,900. Buses incurred the highest expenditure at 113,500 across 355 records, followed by People Movers at 100,600 across 334 records and Mini Buses at 88,800 across 312 records.

Despite recording the fewest bookings, Buses generated the highest maintenance expenditure. This suggests that MonCity should investigate the maintenance requirements of its Bus fleet. However, differences in vehicle utilisation, fleet size and age must be considered before drawing conclusions about cost efficiency.

Finding 3: Accident Distribution by Damage Severity

Of the 1,000 car-accident records, 373 (37.3%) involved no vehicle damage, representing the largest category. However, 177 records (17.7%) involved severe damage.

Very minor damage accounted for 155 records (15.5%), minor damage for 148 (14.8%) and moderate damage for 147 (14.7%).

Although no-damage incidents were most frequent, the substantial number of severe-damage records highlights the importance of investigating accident circumstances and vehicle system failures.

Finding 4: Accident Distribution by Zone

ZoneC recorded the highest number of car-accident records (261), followed by ZoneD (248), ZoneB (247) and ZoneA (244).

However, ZoneD had the highest proportion of severe-damage records at 20.97% (52 out of 248), compared with ZoneC at 17.62%, ZoneA at 17.21% and ZoneB at 14.98%.

These findings indicate that ZoneD deserves particular attention in investigations of severe vehicle damage, despite ZoneC recording the highest overall number of incidents.

Finding 5: Accident Analysis by Error Code

Error002 (Low Battery) was associated with the highest number of car-accident records (373), but none involved severe vehicle damage.

In contrast, Error001 (Image recognition system failed) was associated with 169 records, including 51 severe-damage cases. Its severe-damage proportion was approximately 30.18%, the highest among the five reported error codes.

Error003 (front sensor signal loss) was associated with 163

records and 43 severe-damage cases, while Error004 (Lidar system failure) was associated with 145 records and 42 severe-damage cases.

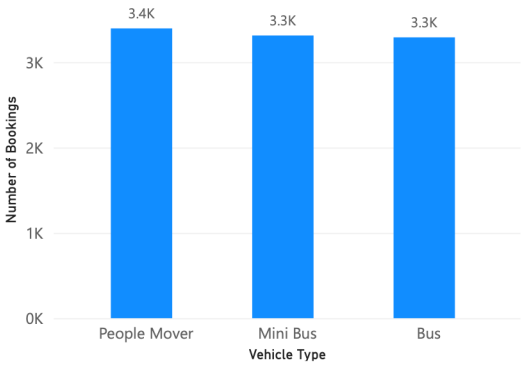
These results suggest that image recognition, sensor and Lidar-related errors warrant further technical investigation. However, the analysis identifies associations rather than establishing that these system errors directly caused the accidents.

Analytical Limitations

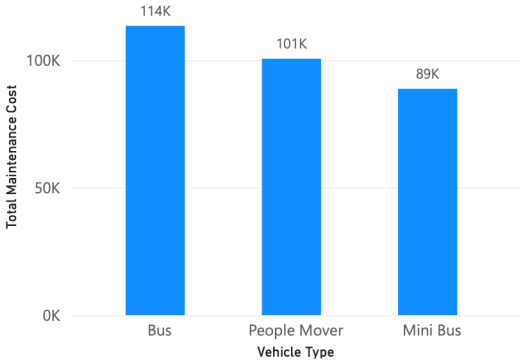
The analysis is based on aggregated historical records. Booking counts do not represent actual passenger occupancy, and accident-record counts do not represent exposure-adjusted accident rates.

MonCity — Operational Performance & Safety Dashboard

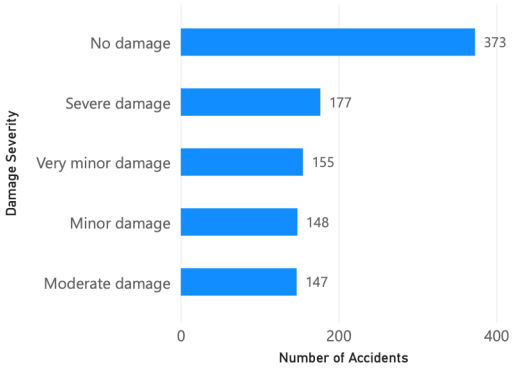
Booking Demand by Vehicle Type



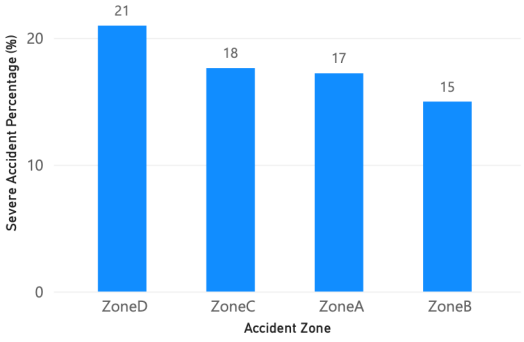
Total Maintenance Cost by Vehicle Type



Accident Distribution by Damage Severity



Severe Accident Percentage by Zone



Accident Distribution by Error Code

